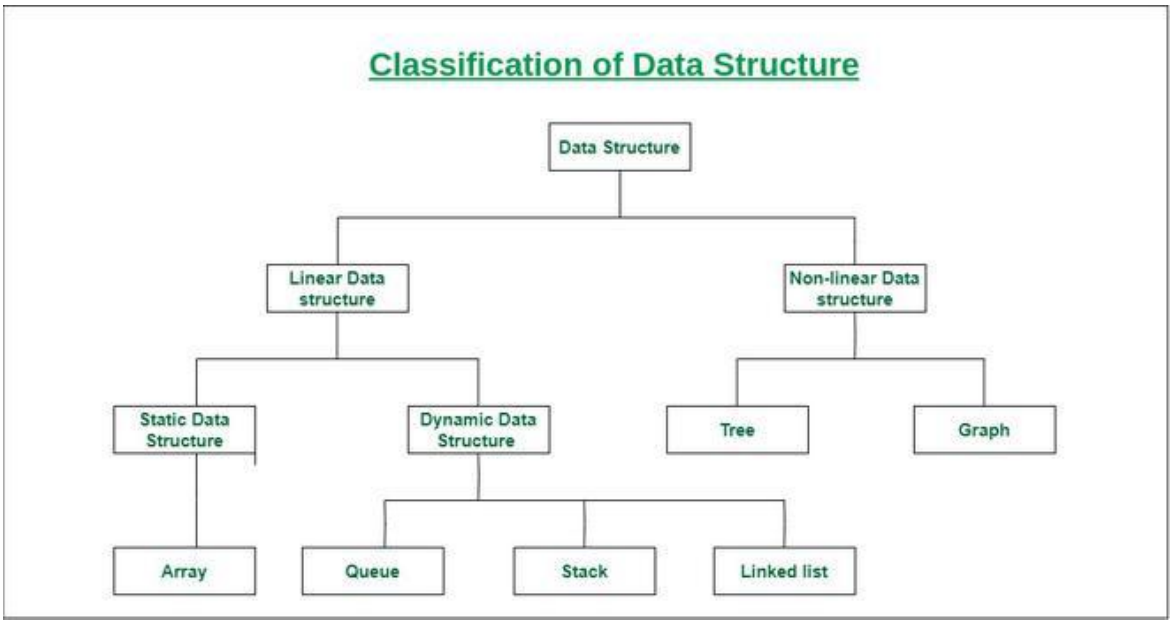
Introduction of Data Structure

A data structure is a specialized format for organizing, processing, retrieving, and storing data items in a storage device or computer's memory. It provides an efficient way to organize data into proper format. Data Structures are generally based on the ability of a computer to fetch and store data at any place in its memory specified by an address. It is the structure representation of logical relationship between elements of data. Data structure can be defined as triple (D,F,A):

- D is data,
- F is a set of function and
- A is a set of axioms.

Classification of Data Structure



Data structure is generally classified into two main categories: Linear Data Structure and Non-Linear Data Structure

1. **Linear Data Structure:**

When elements or data items are arranged in a sequential order, i.e., one by one, where each element is connected to its previous and next adjacent elements, such a data structure is called a linear data structure.

However, a linear data structure is also classified into sub-categories.: Static and Dynamic Data Structure

- **Static data structure:** Static data structure has a fixed memory size. It is easier to access the elements in a static data structure. *Example: array .*

- **Dynamic data structure:** In the dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code. *Examples: stack and queue.*

2. **Non-linear Data Structure:**

When elements are not arranged sequentially or linearly, but instead are organized in a hierarchical or network structure where an element can be connected to multiple other elements, such a data structure is called a non-linear data structure. In a non-linear data structure, we can't traverse all the elements in a single run. Example: tree, graph, and hash table.

Operations on Data Structure

Creation: Process of creating a data structure.

Insertion: Process of inserting an element to a data structure.

Deletion: Process of removing an element from a data structure.

Traversing: Process of visiting each and every element (data items) of a data structure.

Searching: Process of finding a data items in a data structure.

Sorting: Process of arranging data items either in ascending or descending order.

Concatenation: Process of appending two data structure.

Display: Process of showing or displaying data items of a data structure.

Destroying: This operation is applied at the last of data structure, when the data structure is no more needed. It is a process of collapse of a data structure.

Implementation of Data Structure

Static Implementation:

In this implementation of a data structure, an array is used. The size of the data structure cannot be changed while writing the program. Once the size is defined during program writing, it remains fixed.

Dynamic Implementation:

In this implementation of a data structure, pointers are used. It is basically done during the runtime of the program. We can define the size of the data structure as per requirement for specific purposes. We can extend or reduce the size of the data structure to store elements.

Introduction of Abstract Data Type

Abstract Data Type is better known as ADT. An abstract data type is a mathematical model or concept that defines a data type logically. It specifies a set of data and a collection of operations that can be performed on that data. It provides the essential features with which the user can interact, but it does not specify the physical (internal) implementation details.

The definition of an ADT only mentions what operations are to be performed, but not how these operations will be implemented. It focuses on the “what” rather than the “how.” It is called abstract because it gives an implementation-independent view.

The main concept is abstraction in an ADT, which hides the complexity from the user and represents only the essential features. ADT is just a theoretical concept that is used in the design and analysis of algorithms. Example: stack, queue, list, set and map.

The user does not know how the data types are implemented but they only know that a data type defines the type of variable that can hold a particular type of value. For example, we use data types like int, float, and char, which define the type of a particular variable that can store a specific type of value. However, we do not know how these data types are implemented or how the variables are physically allocated memory to store the value. So, the user only needs to know what a data type can do, not how it does it. Now, we’ll define three ADTs namely list ADT, Stack ADT and Queue ADT.

Linear Data Structure VS Non-Linear Data Structure:

Data Structure VS Abstract Data Type: